

Día D — qué ejecuta cada uno

Comandos exactos, en orden. Se copian tal cual; lo **resaltado** es lo único que se cambia.

Antes del sábado — la línea base en `main`

✓	Quién	Qué queda hecho
☐	Mateo	fase 2 mergeada en <code>main</code> : las pantallas hablando por <code>fetch</code> .
☐	Johan	fase 3 mergeada: <code>lib/db.js</code> , <code>lib/higgsfield.js</code> y las cuatro rutas bajo <code>app/api/</code> .
☐	Tomás	T030 primero . Las ocho llaves (Clerk, MongoDB Atlas, Vercel Blob, Higgsfield) en <code>.env.local</code> y en Vercel → <i>Project Settings</i> → <i>Environment Variables</i> .
☐	Santiago	fase 5: merge, <code>npm test</code> , <code>npm run build</code> y T039 — desplegar en Vercel . URL: https://ia-generator-openspec.vercel.app
☐	Santiago	<code>npm run smoke</code> https://ia-generator-openspec.vercel.app en verde.
☐	Todos	Recorrido a mano en el celular sobre la URL pública: entrar → foto → imagen → video → descargar.

20 minutos antes de clase

Cada uno, en su máquina, dentro de `speckit-ai-generator`:

```
git checkout main
git pull origin main
npm install
npm test
```

Tiene que decir `0 failed`. Si sale rojo, avisa al grupo **ahora**.
Deja tu agente abierto en esa carpeta.

- **Santiago** — `npm run smoke` <https://ia-generator-openspec.vercel.app>, abre la URL en el celular y deja el panel de Vercel en una pestaña.
- **Tomás** — confirma que su `.env.local` con las ocho llaves sigue en su máquina.
- **Sergio** — `spec.md` y `tasks.md` abiertos y proyectando.
- **Todos** — hotspot del celular encendido.

Cada quien corre su ciclo completo en **su máquina** y sobre **su archivo de spec**.

1 Sergio registra la HU y reparte

≈ 2 min · main

```
git checkout main
git pull origin main
```

/speckit-specify Agrega dos User Stories nuevas al final de la seccion User Scenarios de specs/001-ai-media-generator/spec.md, numeradas User Story 5 y User Story 6.
 HU 5: "historia literal que dictó el profesor"
 HU 6: "segunda historia literal"
 Cada una con sus Acceptance Scenarios numerados en formato Given/When/Then.
 NO crees una feature nueva. NO corras create-new-feature.sh. NO cambies de rama.
 NO toques plan.md ni tasks.md. Solo edita ese spec.md que ya existe.

```
git add specs/001-ai-media-generator/spec.md
git commit -m "HU en vivo: User Stories 5 y 6 dictadas en clase"
git push origin main
```

Reparte en voz alta — un archivo, un dueño. Mira los Acceptance Scenarios:

- Archivos **nuevos** → los tres en paralelo: **Mateo** pantalla, **Johan** ruta de API, **Tomás** consulta o reporte.
- Pantalla existente + ruta de API** → dos personas: una app/.../page.jsx, otra app/api/...
- Un solo archivo** → **una sola persona** hace esa HU completa; los otros toman la otra HU.

Di qué le toca a cada uno, qué archivos son suyos, y **el nombre de su rama**: feature/categorias, feature/exportar, feature/auditoria. Si sobra trabajo, lo toma **Santiago**.

2 Mateo, Johan y Tomás — cada uno en su máquina

≈ 6 min

2.1 — Tu rama, con el nombre que dijo Sergio:

MATEO

```
git checkout main
git pull origin main
git checkout -b feature/___
```

JOHAN

```
git checkout main
git pull origin main
git checkout -b feature/___
```

TOMÁS

```
git checkout main
git pull origin main
git checkout -b feature/___
```

2.2 — Tu Spec, Plan y Tasks. Escribe **tu nombre** en la ruta:

Escribe specs/001-ai-media-generator/hu-viva/tu-nombre.md, y solo ese archivo.

Historia: "la parte de la HU que me tocó, literal"

El archivo lleva cuatro secciones:

- Historia — la frase de arriba, textual.
- Criterios de Aceptacion — numerados CA-1, CA-2, CA-3, en formato Dado / Cuando / Entonces.
- Plan — la lista exacta de archivos que vas a crear o modificar, y por que cada uno.
- Tasks — numeradas, y cada una dice que CA cubre.

Reglas: no toques spec.md, plan.md ni tasks.md. Si para cumplir la Historia necesitas modificar un archivo que ya existe y no esta en tu Plan, parate y dimelo antes de escribir.
 Persistencia real: si la Historia guarda datos, van a MongoDB por lib/db.js, nunca en memoria.

2.3 — Implementa test-first. Lee tu Plan antes de lanzarlo:

Implementa specs/001-ai-media-generator/hu-viva/tu-nombre.md, test-first:
 por cada Criterio de Aceptacion, primero la prueba que falla y despues el codigo que la pasa.

Nombra cada prueba empezando por su criterio: it("CA-2: ...").
 Toca unicamente los archivos de la seccion Plan de ese documento. Ninguno mas.
 Al terminar corre npm test y dejalo en verde.

2.4 — Pruebas, commit y push:

```
npm test
git add -A
git commit -m "feat: la historia en cinco palabras (CA-1 a CA-N)"
git push -u origin feature/___
```

Si npm test sale rojo, no hagas push. Pídele al agente: «la prueba CA-2 está fallando, arrégla y vuelve a correr npm test». Cuando tu push termine, anúncialo en voz alta — **Santiago** no arranca hasta oír los tres.

3 Santiago integra y despliega

≈ 2 min · main

Solo cuando los tres hayan subido:

```
git checkout main
git pull origin main
git fetch origin
```

Un merge por línea, esperando a que termine cada uno:

```
git merge --no-ff origin/feature/
git merge --no-ff origin/feature/
git merge --no-ff origin/feature/
```

Con los tres adentro:

```
npm test
npm run build
git push origin main
```

Ese push dispara el despliegue en Vercel solo. Nadie corre un deploy a mano. Mientras construye, proyecta el grafo:

```
git log --graph --oneline -15
```

4 Todos validan sobre el despliegue

≈ 1 min

1. **Santiago** proyecta Vercel y espera el check verde (1–2 min).
2. `npm run smoke` <https://ia-generator-openspec.vercel.app>
3. Abre <https://ia-generator-openspec.vercel.app> en el celular y recorre la HU nueva de punta a punta.

Si el deploy no ha terminado: `npm run dev` y muestra `http://localhost:3000` mientras tanto, pero **vuelve a la URL pública** apenas esté verde.

Si algo se rompe

Qué pasó	Quién	Qué ejecuta
Conflicto en el merge	Santiago	En <code>package.json</code> : dejar las dependencias de los dos lados. En <code>lib/*.js</code> : dejar la implementación real y borrar el stub. Luego <code>git add -A</code> y <code>git commit</code> .
El merge se enredó	Santiago	<code>git merge --abort</code> y reintentar solo con esa rama.
<code>npm test</code> en rojo	Quien lo corrió	No hacer push. Decirle al agente qué CA falla, y volver a correr <code>npm test</code> .
<code>npm run build</code> en rojo	Santiago	No hacer push a <code>main</code> . Arreglar solo lo que rompió el merge y volver a construir.
El deploy de Vercel falla	Santiago	Abrir el log: casi siempre falta una variable de entorno. Agregarla en <i>Project Settings</i> y <i>Redeploy</i> . Mientras tanto <code>npm run dev</code> .
El agente toca archivos de otro	El dueño	Pararlo: « <i>revierte eso, solo puedes tocar los archivos de la sección Plan de mi spec</i> ».
El agente se contradice	Sergio	<code>/speckit-analyze</code> — no cambia nada, muestra las contradicciones.
Se cayó el wifi	Todos	Hotspot del celular. Las pruebas corren sin red; el deploy y la demo no.

Reglas que no se rompen

1. Es `/speckit-implement`, **no** `/implement`. Y nunca sin argumento.
2. Primero el spec, después el código.
3. Una prueba por criterio de aceptación, nombrada `CA-N`, escrita antes del código.
4. Nadie escribe fuera de la sección **Plan** de su propio spec. Un archivo, un dueño.
5. Los merges van uno por uno y con `--no-ff`.
6. `git push origin main` solo lo hace **Sergio** en el paso 1 y **Santiago** en el paso 3.